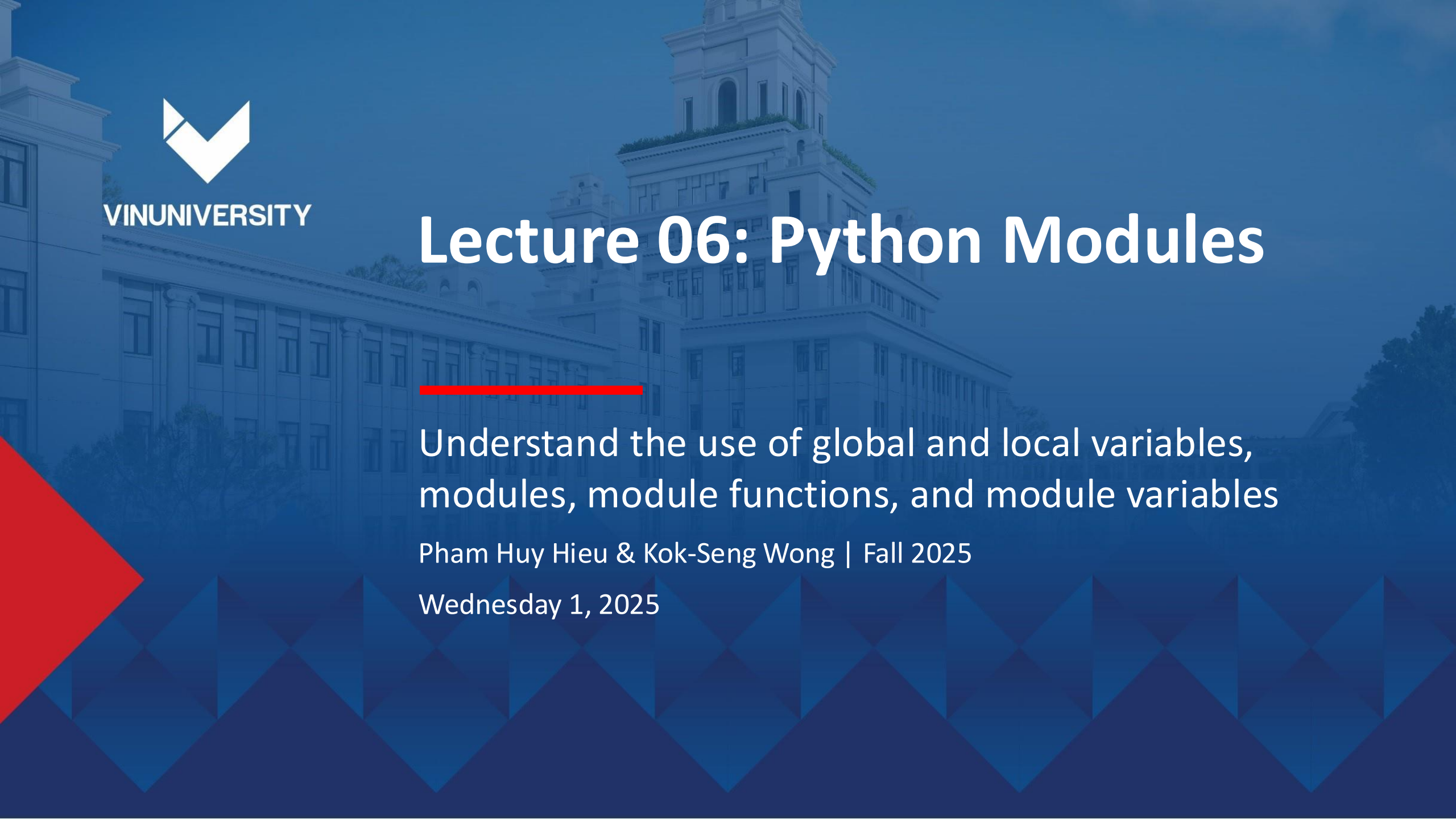


Lecture 06: Python Modules

Understand the use of global and local variables, modules, module functions, and module variables

Pham Huy Hieu & Kok-Seng Wong | Fall 2025

Wednesday 1, 2025

LECTURE 05 (HOMEWORK)

Identify and correct the errors (if any)?

```
def calculate_sum(a, b):
    result = a + b
    return result

print(calculate_sum(3, 5))
```

```
def greet(name):
    return "Hello, " + name

print(greet("Alice"))
print(greet("Bob"))
```

```
def find_max(numbers):
    max_num = max(numbers)
    return max_num

numbers = [4, 9, 2, 7, 5]
print(find_max(numbers))
```

LECTURE 05 (HOMEWORK)

Identify and correct the errors (if any)?

```python
def calculate_sum(a, b):
    result = a + b
    return result

print(calculate_sum(3, 5))
```

Correct! No errors here!

```python
def greet(name):
    return "Hello, " + name

print(greet("Alice"))
print(greet("Bob"))
```

Correct! No errors here!

```python
def find_max(numbers):
    max_num = max(numbers)
    return max_num

numbers = [4, 9, 2, 7, 5]
print(find_max(numbers))
```

Potential confusion but not an error

LECTURE 06: MOTIVATIONAL EXAMPLE

The world's first home robotic chef can cook over 100 meals



[source]

LEARNING OUTCOMES

Upon the completion of this lecture, students will be able to:

- understand global and local variables.
- understand Python modules and how to access a module.
- use built-in and user-defined functions from a module.
- design a module and demonstrate how to import it into a program.

GLOBAL AND LOCAL VARIABLES

Global Variables:

- They are defined and declared outside a function.
- We can use them inside a function.

Local Variables:

- The visibility is only inside the function definition.
- They are available only while the function is executing.

GLOBAL AND LOCAL VARIABLES

```
name = ”CECS"                  # global scope

def display_name():
    name = ”CBM"               # local scope
    print(name)

display_name()
```

GLOBAL AND LOCAL VARIABLES

```
name = ”CECS"                  # global scope

def display_name():
    name = ”CBM"               # local scope
    print(name)

display_name()
```

- If you call the function `display_name()`, the output will be: CBM
- If you just run the code without calling the function, nothing will be printed.

LOCAL VARIABLE

```
def hello():
    temp = "Hi"
    return temp

print(temp)
```

Local variable

Attempt to access local variable outside the function

ACCESSING GLOBAL VARIABLE

- When a local variable is not defined, the global variable will be used.

Global variable

```
def greeting():
    print(m)

m = "Merry Christmas"
greeting()
```

```
def greeting():
    m2 = "Happy New Year"
    print(m1, m2)

m1 = "Merry Christmas"
m2 = "Happy Birthday"
greeting()
print(m2)
```

Local variable

Global variables

CLASS PRACTICE

Sample Code:

```
base = 5
def areaTriangle1(height):
    area = 0.5 * base * height
    return area

areaTriangle1(10)
```

Output:



CLASS PRACTICE

Sample Code:

```
area = 100
def areaTriangle2(base, height):
    area = 0.5 * base * height
    return area

areaTriangle2(5, 10)
```

Output:



CLASS PRACTICE

Sample Code:

```
def f1():
    print(s)

s = "I'm Ali Baba's Friend"
f1()
```

Output:

CLASS PRACTICE

Sample Code:

```
def f2():
    s = "I'm Ali Baba's Friend"
    print(s)

s = "Me too!"
f2()
print(s)
```

Output:



CLASS PRACTICE

Sample Code:

```
def f3():
    print(s)
    s = "I'm Ali Baba's Friend"
    print(s)

s = "Me too."
f3()
print(s)
```

Output:

MODIFYING GLOBAL VARIABLE

global keyword: used to create a global variable and make changes to the variable in a local context.

```
def f4():

    print(s)
    s = "I'm Harry Potter's Friend"
    print(s)

s = "I'm Ali Baba's Friend"
f4()
print(s)
```

demo

CLASS PRACTICE

Sample Code:

```
a = 1

def f():
    print('Inside f():', a)

def g():
    a = 2
    print('Inside g():', a)

def h():
    global a
    a = 3
    print('Inside h():', a)

print('global:', a)
f()
print('\nglobal:', a)
g()
print('\nglobal:', a)
h()
print('\nglobal:', a)
```

Output:



CLASS PRACTICE

What will be the output?

```
k = 50
def func_A():
    k = 100

    def func_B():
        global k
        k = 250

    print("k in function [before calling func_B]: ", k)
    func_B()
    print("k in function [after calling func_B]: ", k)

print("k in main [before calling func_A]: ", k)
func_A()
print("k in main [after calling func_A]: ", k)
```

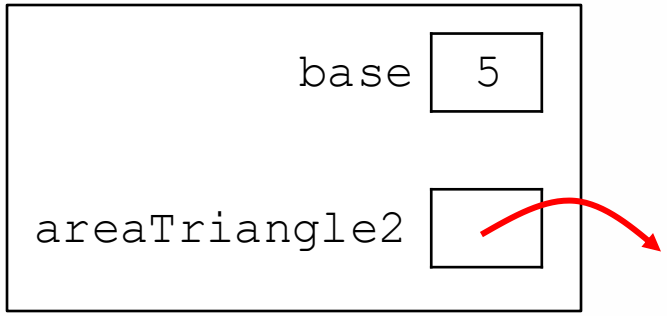
FUNCTION ACCESS TO GLOBAL SPACE

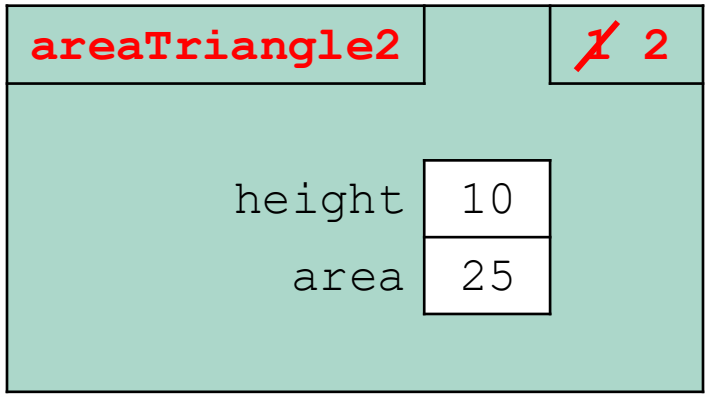
```
base = 5
def areaTriangle2(height):
    area = 0.5 * base * height
    return area

areaTriangle2(10)
```

1
2

Global Space



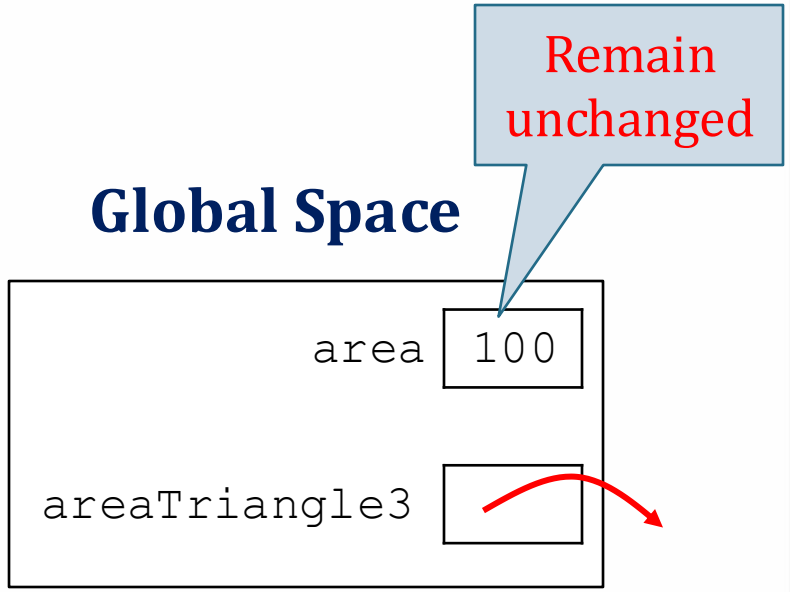


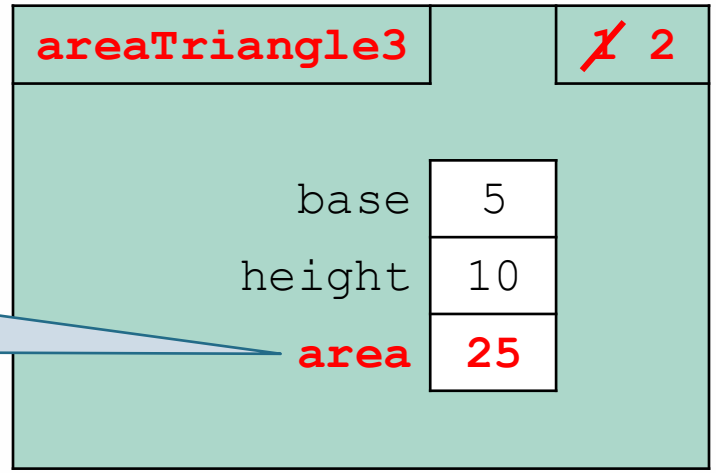
FUNCTION CAN CHANGE GLOBAL SPACE?

```
area = 100
def areaTriangle3(base, height):
    area = 0.5 * base * height
    return area

areaTriangle3(5, 10)
```

1 2





Assignment to a global makes a new local variable

PYTHON NOLOCAL SCOPE AND KEYWORD

- Nonlocal scopes refer to the nearest enclosing function scopes, excluding the global scope.
- To modify variables from a nonlocal scope in a local scope, you use the nonlocal keyword.

```
def outer():
    message = "Hello"

    def inner():
        nonlocal message  # Reference the enclosing variable
        message = "Hello, Python!"  # Modify it
        print("Inside inner:", message)

    inner()
    print("Inside outer:", message)

outer()
```

Output

```
Inside inner: Hello, Python!
Inside outer: Hello, Python!
```



For more: https://www.geeksforgeeks.org/python/python-nonlocal-keyword/

PYTHON NOLOCAL SCOPE AND KEYWORD

Let's see what happens if we remove the nonlocal line.

```
def outer():
    message = "Hello"

    def inner():
        message = "Hello, Python!"  # Creates a NEW local variable
        print("Inside inner:", message)

    inner()
    print("Inside outer:", message)

outer()
```

Output?

PYTHON NOLOCAL SCOPE AND KEYWORD

Let's see what happens if we remove the nonlocal line.

```
def outer():
    message = "Hello"

    def inner():
        message = "Hello, Python!"  # Creates a NEW local variable
        print("Inside inner:", message)

    inner()
    print("Inside outer:", message)

outer()
```

Output

```
Inside inner: Hello, Python!
Inside outer: Hello
```

PYTHON MODULES

PYTHON MODULES

- Logically organize Python code (related code) into a module makes the code easier to understand and use
- A file (.py) containing Python definitions and statements.
- Can contain executable statements as well as function definitions.
- Each module has its own private symbol table, which is used as the global symbol table by all functions defined in the module.
- Read more: https://docs.python.org/3/tutorial/modules.html

MODULES (1/4)

- Modules: library of functions and variables.
- To access a module, we need to import them: `import <module name>`
- To access functions: `<Module name>.<function name>(<arguments>)`

```
>>> import math
>>> x = math.sin(30)
>>> y = math.log10(15)
>>> print(x, y)
-0.9880316240928618 1.1760912590556813
>>> print(math.floor(x), math.ceil(y))
-1 2
```

MODULES (2/4)

- To access variables in a module: `<Module name>.<variable name>`

```
>>> import math
>>> math.pi
3.141592653589793
```

- To access only a selected function in a module:

```
from <module name> import <function name>
```

```
>>> from math import pow
>>> pow(2,3)
8.0
>>> pow(4,2)
16.0
```

Same as x**y?

Re-naming a module

```
>>> import math as m
>>> m.pi
3.141592653589793
```

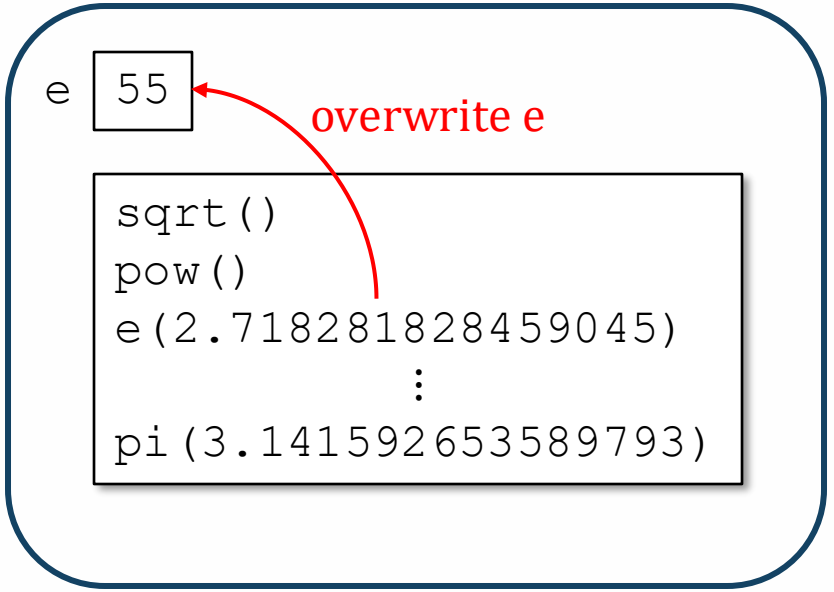
MODULE (3/4)

- To import everything from a module: `from <module name> import *`

- Caution! 

```
>>> e = 55
>>> from math import *
>>> e
2.718281828459045
```

e was overwritten



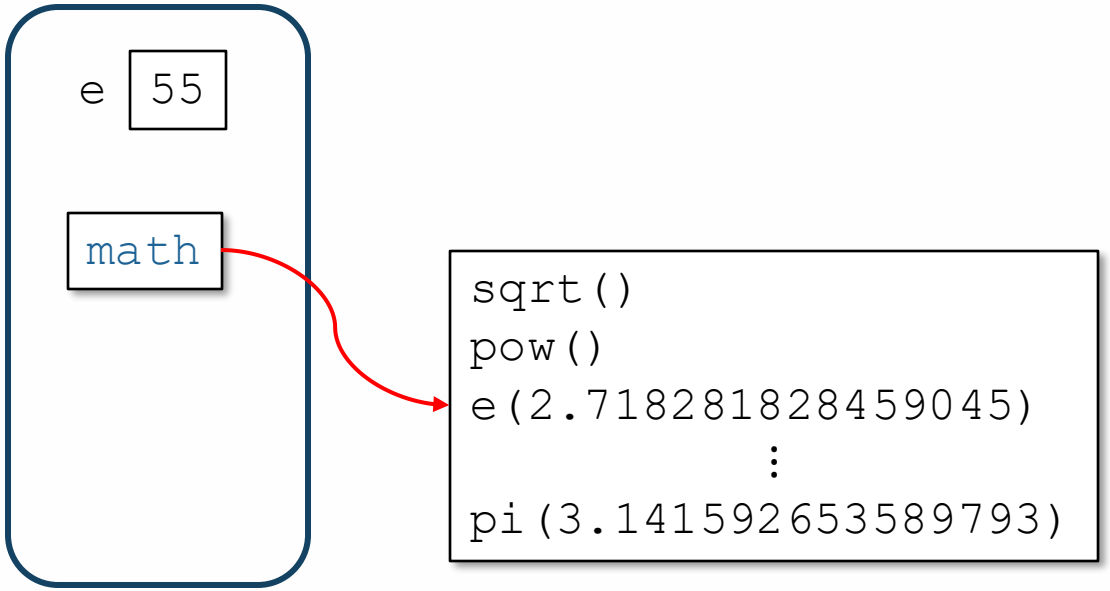
MODULE (4/4)

To avoid overwritten case, we should separate the variables:

```
>>> id(e)
140720974637136
```

```
>>> e = 55
>>> import math
>>> math.e
2.718281828459045
```

```
>>> id(math.e)
1962879648048
```



Every value has an `id`, which is a unique number related to where it is stored in the memory of the computer. The `id` of a variable is the `id` of the value to which it refers.

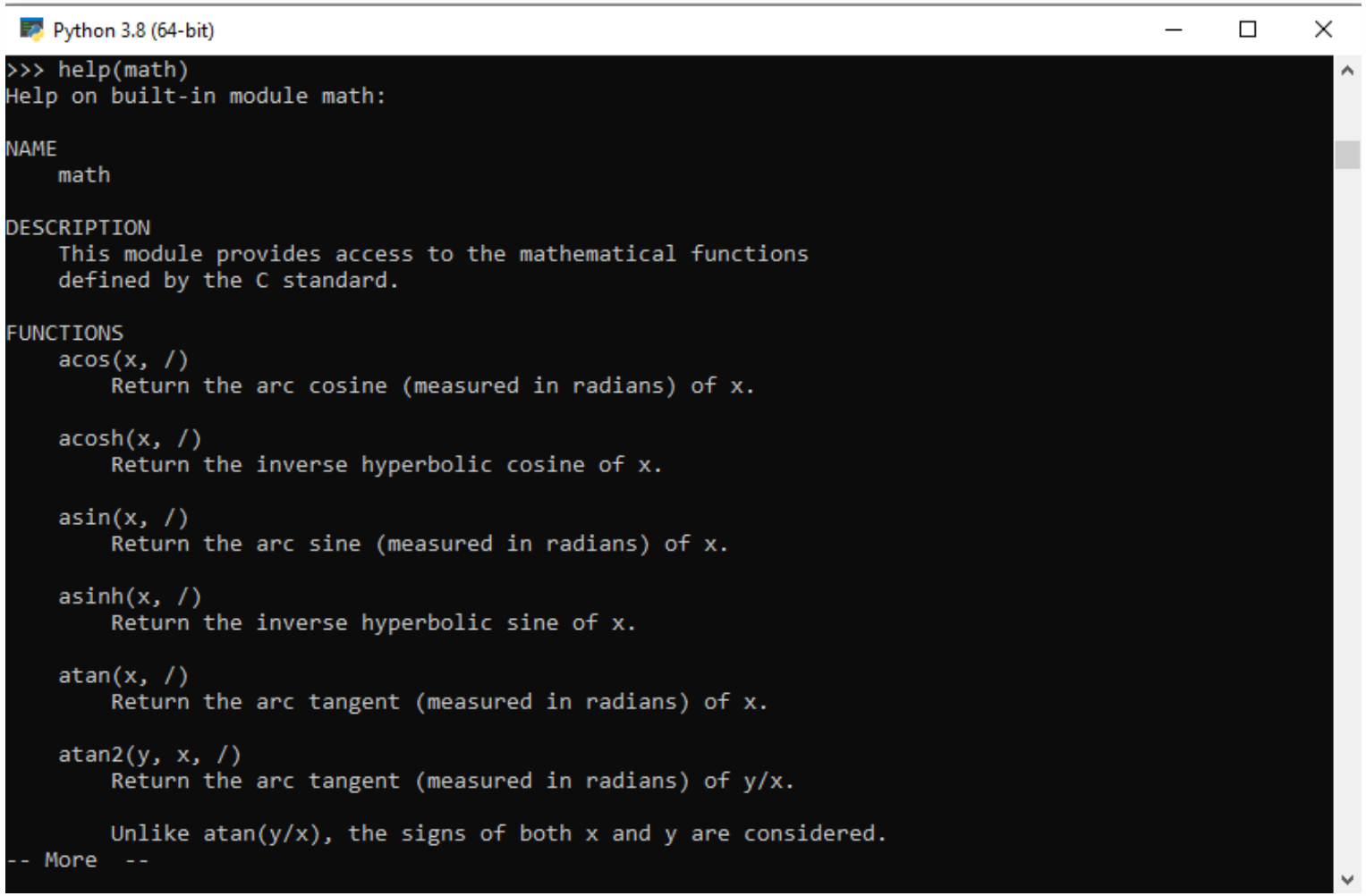
HELP FUNCTION

- To see the library of a module:

```
import <module name>
```

```
help <module name>
```

- To see more, just press <ENTER>
- To quit, just press ‘q’



PYTHON DOCUMENTATION

https://docs.python.org/3/library/math.html

Table of Contents

math — Mathematical functions

- Number-theoretic and representation functions
- Power and logarithmic functions
- Trigonometric functions
- Angular conversion
- Hyperbolic functions
- Special functions
- Constants

Previous topic

numbers — Numeric abstract base classes

Next topic

cmath — Mathematical functions for complex numbers

`math` — Mathematical functions

This module provides access to the mathematical functions defined by the C standard.

These functions cannot be used with complex numbers; use the functions of the same name from the `cmath` module if you require support for complex numbers. The distinction between functions which support complex numbers and those which don't is made since most users do not want to learn quite as much mathematics as required to understand complex numbers. Receiving an exception instead of a complex result allows earlier detection of the unexpected complex number used as a parameter, so that the programmer can determine how and why it was generated in the first place.

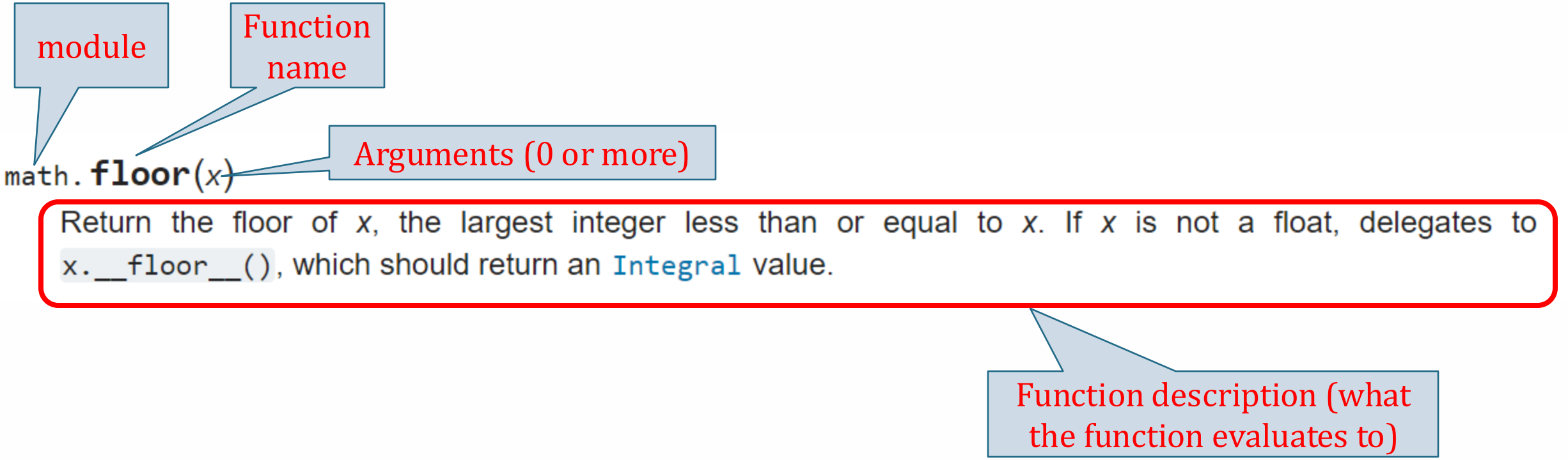
The following functions are provided by this module. Except when explicitly noted otherwise, all return values are floats.

Number-theoretic and representation functions

`math.`**`ceil`**(*x*)

Return the ceiling of *x*, the smallest integer greater than or equal to *x*. If *x* is not a float, delegates to `x.__ceil__()`, which should return an `Integral` value.

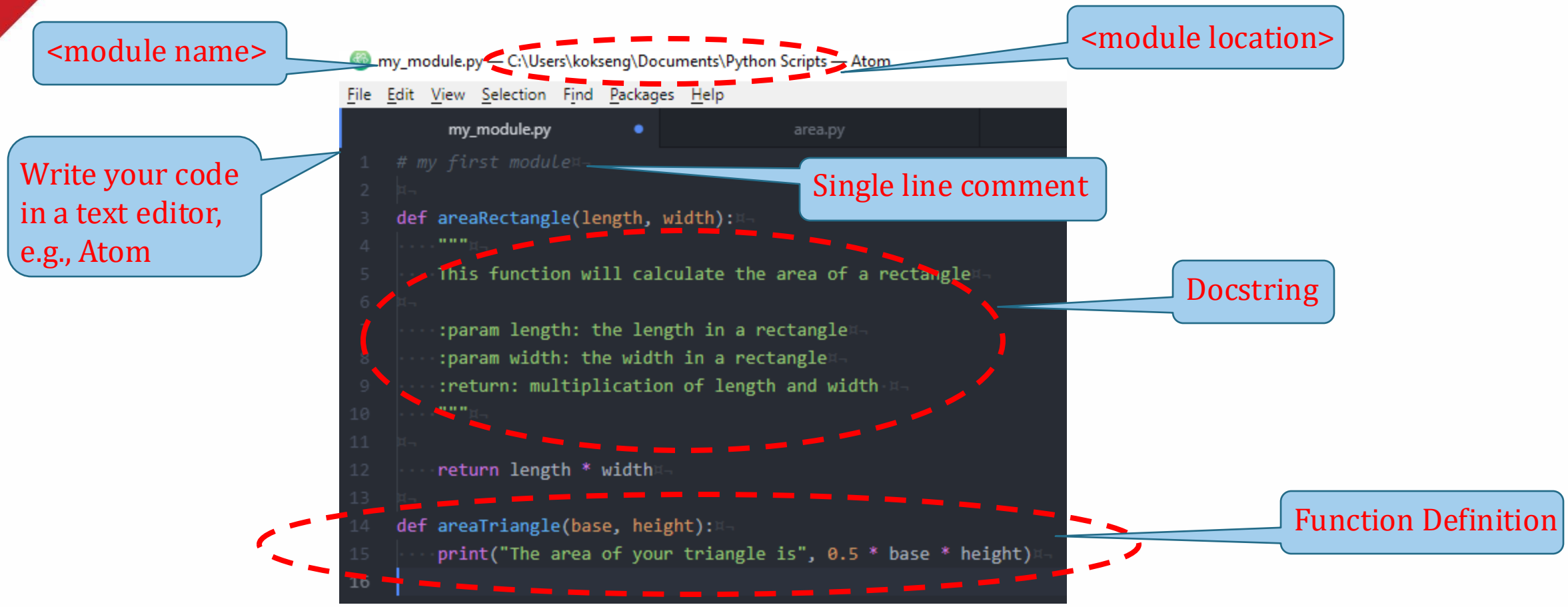
READING THE PYTHON DOCUMENTATION



OTHER USEFUL MODULES

- io: read/write from files
- random: generate random numbers from any distribution
- string: useful string function
- sys: information about your OS

CREATING A MODULE



For other editors, please refer to this link: https://wiki.python.org/moin/PythonEditors

HOW TO USE YOUR MODULE?

my_module.py — C:\Users\kokseng\Documents\Python Scripts — Atom

File Edit View Selection Find Packages Help

my_module.py | area.py

```
1  # my first module
2
3  def areaRectangle(length, width):
4      """
5      This function will calculate the are
6
7      :param length: the length in a rectan
8      :param width: the width in a rectangle
9      :return: multiplication of length and width
10     """
11
12     return length * width
13
14 def areaTriangle(base, height):
15     print("The area of your triangle is", 0.5 * base * height)
16
```

Display the documentation of your module

Anaconda Prompt (anaconda3) - python

```
(base) C:\Users\kokseng\Documents\Python Scripts>
(base) C:\Users\kokseng\Documents\Python Scripts>python
Python 3.7.6 (default, Jan  8 2020, 20:23:39) [MSC v.19[illegible] 64 bit (AMD64)]
:: Anaconda, Inc. on win32
Type "help", "copyright", "credits" or "license" for
>>> import my_module
>>> help(my_module)
Help on module my_module:

NAME
    my_module - # my first module

FUNCTIONS
    areaRectangle(length, width)
        This function will calculate the a

        :param length: the length in a rectangle
        :param width: the width in a rectangle
        :return: multiplication of length and width

    areaTriangle(base, height)

FILE
    c:\users\kokseng\documents\python scripts\my_module.py


>>> my_module.areaRectangle(10, 20)
200
>>> my_module.areaTriangle(14, 8)
The area of your triangle is 56.0
>>>
```

Run python from the same folder as your module

import module (same name as module but without the extension .py)

accessing function in the module

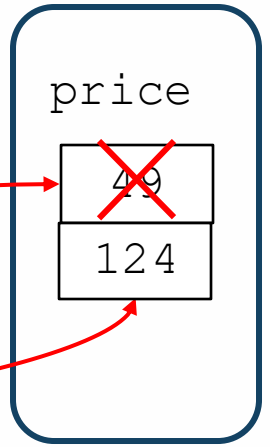
RUNNING A PYTHON SCRIPT

my_script.py — C:\Users\kokseng\Documents\Python Scripts — Atom

File Edit View Selection Find Packages Help

my_script.py

```
1  """
2  This is my script!
3  """
4
5  price = 49
6  price = 75 + price
7
```



Anaconda Prompt (anaconda3)

```
(base) C:\Users\kokseng\Documents\Python Scripts>python my_script.py

(base) C:\Users\kokseng\Documents\Python Scripts>
```

python <script file name>

No output!

PRINT()

- `print( <expression> )`

my_script.py — C:\Users\kokseng\Documents\Python Scripts — Atom

File Edit View Selection Find Packages Help

my_script.py

```
"""
This is my script!
"""

price = 49
price = 75 + price

print(price)
```

`print` evaluates the `<expression>` and writes the value to the console

Anaconda Prompt (anaconda3)

```
(base) C:\Users\kokseng\Documents\Python Scripts>python my_script.py
124

(base) C:\Users\kokseng\Documents\Python Scripts>
```

Output!

THANK YOU FOR YOUR ATTENTION!